

 Model, effort, and what a session costs

[↑](#) [Retour à « Resources »](#)

Model, Effort, and What a Session Costs

Read this once, before Lab 1. Then stop thinking about it.

The fiche statement behind C1 says "*an efficient combination of model and effort level*". This document is that phrase, in full, so that the lecture can spend its ninety minutes on the part that will still be true in three years. Everything here has a two-year shelf life, which is exactly why it is a document you read once rather than a competency you are graded on. Nothing in the assessment grid depends on it. The point is that you stop losing afternoons to it.

The short version

- Pick a top-tier model. Pick **medium** effort. Leave both alone.
- If the output is not good enough, the fix is almost never the model. It is the harness and the environment — the specification, the ticket, the instruction document, the context you did or did not spend.
- Every input token is re-sent on every request. The context discipline you are taught for quality reasons is the same discipline that controls the bill.

If you only take those three lines, you have taken enough.

Effort

Frontier models expose a **reasoning effort** setting — low, medium, high, max, under various names. It buys the model more reasoning tokens before it answers.

Medium is the working setting. It is the sweet spot for ordinary feature work, and the reason to choose it is not that it wins a benchmark but that choosing it lets you stop choosing. Time spent tuning effort per task is time not spent shipping, and the tuning never pays back what it costs.

Three things worth knowing about the top of the range:

- **Max exists for benchmarks.** A provider ships it so it can claim a couple of points over a competitor on a public leaderboard. The quality gain over high, on your work, is marginal; the token cost is not.
- **Higher effort is slower, not faster.** It is a latency trade as much as a quality one.
- **Higher effort spends your context budget.** Reasoning tokens are tokens. Raising effort moves you toward the dumb zone sooner — which means the setting interacts with everything Lecture 1 teaches about decomposition. A high-effort session fits less work than a medium-effort one.

The one place to raise it deliberately: a hard, self-contained question where you have already decided the session is short — a gnarly diagnosis, a design decision you are grilling. Raise it, ask, drop back.

Choosing a model

Once you are on a top-tier model, the marginal return from swapping to whichever model topped last week's benchmark is close to zero. The model is roughly half the picture, and it is the half you cannot improve. Work you put into the harness and the environment — your specification, your tickets, your **CLAUDE.md**, your hooks — survives a model change. Work you put into tuning one model does not.

So: **rotating models weekly is a way of never getting good at any of them.** Pick one, learn how it fails, build your steering around that.

If you genuinely need to settle an argument about which model is better *for your codebase*, the only honest method is an A/B: give two setups the same task, from the same starting context, and judge the outputs. Repeat it three or four times. That produces data about your work. A benchmark graph produces data about somebody else's.

On the cost axis, the price per token is the wrong number. What matters is:

- **Cost per task.** A model that burns a lot of cheap tokens can be more expensive than one that burns few expensive ones, and vice versa. Efficiency is what you get for the tokens, not what the tokens cost.
- **Which pricing you are actually on.** A subscription makes the same model dramatically cheaper than API rates for the same work. Comparing a subscription model against an API model on list price compares nothing.

What a session costs

You do not need to be able to compute a bill. You need the mechanism, because the mechanism is why the context discipline is worth having.

A session grows by appending, and the whole thing is re-sent every request. This is the same fact as the "context is re-uploaded" slide in Lecture 1, seen from the invoice. A token you put into the window on the first request is paid for again on the second, the third, and every one after it. Sediment in a **CLAUDE.md** is not paid for once; it is paid for on every turn of every session for as long as the line exists.

That is the whole argument. Quality and cost are the same argument, and if you get the quality one right the cost one comes free.

Prefix caching is the mechanism that makes long sessions survivable. Providers cache the unchanged prefix of your request — system prompt, tool definitions, everything up to your newest message — and bill reads against it at roughly a tenth of the base input rate. Cheap, not free: a cache read still appears on the bill.

Two consequences that surprise people:

- **The cache expires.** Send a message, take an hour's lunch, send another, and the next message bills far more than the ones before the break — the prefix timed out, so the entire history is charged at the base rate again. The session did not restart; only the discount did.
- **Anything you change early in the context invalidates everything after it.** Which is a second reason the front of your context wants to be stable and small.

Output and reasoning tokens are billed the same way as each other, and at a much higher rate than input. This is the other half of why max effort is a poor default.

Seeing it for yourself

Two tools, both already in this folder, both worth twenty minutes:

- [Showing context in the status line](#) — put the live token count in your status line. Once the number is visible, the boundary decisions in Lecture 1 stop being theoretical.
- [Killing bloat](#) — read the actual payload your harness sends before you type anything, and cut it. A typical starting context of 68k tokens comes down to under 20k, and every one of those tokens was being re-sent, every request, all week.

/context in Claude Code gives you the same breakdown in one screen. **/usage** tells you what is left of your plan. Neither removes anything; they only show you.

Where this connects to the graded work

Nowhere directly, and that is deliberate. But two links are worth holding:

- **Effort spends context**, so it is a decomposition variable. If you raise effort, your units of work get smaller (C1).
- **The cheapest token is the one you never send.** Every C2 decision about pushing a document into **CLAUDE.md** versus pointing at it from disk is a recurring cost decision as well as an attention decision.

The trap to avoid

The failure mode this document exists to prevent is **min-maxing**: fiddling with model and effort in the belief that the right combination is what stands between you and good output. It is not. When an agent underperforms on your project, the leverage is in the harness and the environment, near enough every time — and that is the entire subject of this course.

Set it once. Get on with the work.

Credits

aihero.dev (Matt Pocock) — the *Concepts* section of the [AI Coding Crash Course](#), in particular the lessons on effort, choosing a model, and cost.